

Verifiable and Fairness-Aware Secure Aggregation for Federated Learning

Abhijat Sharma¹[0000–0003–3815–0541] and Kalikinkar
Mandal¹[1111–2222–3333–4444]

University of New Brunswick, Fredericton, N.B, Canada
`abhijat.sharma@unb.ca`

Abstract. Federated learning (FL) enables collaborative model training without centralizing clients’ private data, but secure aggregation alone does not ensure that submitted updates are valid, correctly included in the aggregate, or auditable for systematic participation disparities. Existing approaches address these concerns separately: lattice-based secure aggregation can provide post-quantum protection for model updates, while zero-knowledge techniques can enforce constraints on client inputs, but the resulting aggregate is still difficult for clients to independently audit for correct inclusion and group-level participation.

We present a privacy-preserving federated learning protocol that combines ring-learning-with-errors (RLWE) secure aggregation with zero-knowledge input validation, Pedersen commitments, and a proof-of-inclusion (PoI) mechanism. Each client encrypts its model update under an RLWE-based construction and proves in zero knowledge that the encrypted representation satisfies the prescribed validity relation. The server can therefore aggregate only validated updates without learning individual model parameters. We further introduce a PoI mechanism that allows clients to verify consistency between their individual contribution and the claimed aggregate. Building on this mechanism, we formulate fairness-aware inclusion auditing using verifiable group-level participation and contribution statistics, allowing disparities in client inclusion to be detected without revealing individual updates or sensitive group information.

We implement the cryptographic components using optimized RLWE arithmetic and number-theoretic transforms (NTT), Pedersen commitments, and Bulletproof R1CS proofs, with integration into a federated learning implementation. We evaluate the computational and communication overheads of the resulting construction, including proof generation and verification, proof size, and proof-of-inclusion costs, and evaluate the resulting inclusion and model-level fairness measures. Our results characterize the trade-off between R1CS representations for efficient zero-knowledge validation and demonstrate how cryptographically verifiable inclusion can extend secure aggregation from privacy-preserving computation toward auditable and fairness-aware federated learning.

Keywords: Federated Learning · Secure Aggregation · Ring-LWE · Zero-Knowledge Proofs · Bulletproofs · Pedersen Commitments · Proof of Inclusion · Privacy-Preserving Machine Learning · Fairness Auditing

1 Introduction

Contents (overview):

- Intro to PPFL and secure aggregation
- Fairness requirement: Group fairness, individual fairness
- Federated Learning (with RLWE-based encryption and Pedersen commitments)
- Proof of inclusion (to ensure auditable group fairness)
- Secure aggregation by masking
- Full protocol outline
- Proof overview of protocol security, including privacy and fairness guarantees

Federated learning (FL) enables a collection of clients to collaboratively train a machine-learning model while retaining their local training data. Rather than transferring raw examples to a central service, each client computes a local model update and contributes that update to an aggregation procedure. This architecture has become an important paradigm for distributed learning over data that is naturally decentralized or subject to privacy constraints. However, retaining raw data locally does not by itself prevent privacy leakage: model updates can contain sensitive information about the underlying training examples, while an untrusted aggregation server may attempt to recover individual client contributions from the messages it receives.

Secure aggregation (SA) addresses this problem by allowing an aggregation server to learn an aggregate of client inputs without learning the individual inputs. In its canonical form, the server obtains

$$M = \sum_{i=1}^n m_i$$

while individual vectors m_i remain hidden. Early practical constructions established the feasibility of secure aggregation at substantial scale, while subsequent work has focused on reducing communication, supporting repeated FL rounds, tolerating client dropouts, and strengthening the adversarial model. In particular, the single-server construction of Bell et al. demonstrated logarithmic-overhead secure aggregation, while later systems such as Flamingo targeted the repeated-round setting inherent to FL.

Our Contribution. Our contributions are as follows.

1. *RLWE-based privacy-preserving aggregation with input validation.* We design an RLWE-based secure aggregation protocol for federated-learning updates in which individual client contributions remain hidden from the aggregation server while validity of the submitted contribution can be established cryptographically.

2. *Zero-knowledge validation of the RLWE relation.* We formulate the client validity condition as an R1CS relation and instantiate its proof using Bulletproofs and Pedersen commitments. The construction binds the committed client input to the corresponding RLWE computation without revealing the private values used to generate the ciphertext.
3. *Aggregate consistency and proof-of-inclusion mechanism.* We incorporate an explicit consistency check between the aggregate ciphertext, individual client contributions, commitments, and the reconstructed aggregate relation. This enables a client to verify that its contribution is included consistently in the server’s aggregate.
4. *Privacy-preserving fairness evaluation.* We formulate an extension in which group-level statistics can be committed to and, where required, proven without revealing individual client updates or sensitive group membership. Downstream model fairness is evaluated using established metrics including statistical parity difference, equal opportunity difference, equalized-odds disparity, and worst-group performance.
5. *Efficient hybrid implementation.* We implement the machine-learning and system components in C++/LibTorch and the security-critical cryptographic components in Rust using curve25519-dalek and bulletproofs. The interface is designed around fixed-size encoded buffers and explicit integer representations rather than exposing cryptographic internals to the ML layer.
6. *R1CS representation and performance analysis.* We experimentally compare an NTT/vector R1CS representation against an RLWE-matrix representation. The evaluation demonstrates a concrete trade-off between proof size and computational cost, showing that exploiting public RLWE coefficients as linear-combination coefficients can substantially reduce proof size.
7. *Security analysis.* We formalize privacy, input-validation soundness, aggregate correctness, and aggregation-consistency properties under explicitly stated assumptions on the RLWE encryption layer, Pedersen commitments, the Bulletproof proof system, and the communication/authentication layer.

The combination of these contributions positions the work not as a replacement for highly optimized general-purpose secure aggregation systems, but as an investigation into a post-quantum, verifiable secure-aggregation architecture in which the RLWE algebra itself is integrated with zero-knowledge input validation.

2 Motivation and Related Work

Privacy of the aggregate, however, is only one requirement for secure federated learning. A malicious client can submit an arbitrary update while remaining hidden from the aggregation server. Such an update may have disproportionately large norm or otherwise violate the constraints expected by the learning algorithm, potentially influencing the global model. This motivates input validation: before accepting a client contribution, the aggregation system should

establish that the encrypted or secret-shared input satisfies a prescribed relation without revealing the input itself. RoFL [2] demonstrated the importance of this direction by combining secure aggregation with privacy-preserving constraints on client updates, including L_2 and L_∞ bounds.

A closely related line of work has subsequently investigated how to make such validation efficient at scale. Bell et al.’s RLWE-SecAgg and ACORN [1] protocols are particularly relevant. RLWE-SecAgg replaces earlier pseudorandom-generator-based encodings with a lattice-based construction, reducing the client computation from $O(\ell \log n)$ to $O(\ell + \log n)$ and the server computation from $O(n\ell \log n)$ to $O(n(\ell + \log n))$, where n denotes the number of clients and ℓ the input dimension. ACORN further incorporates zero-knowledge input validation and reports practical validation overheads for high-dimensional inputs. These results demonstrate that secure aggregation and privacy-preserving input validation need not be treated as fundamentally separate components.

At the same time, recent work has explored alternative trust and verification architectures. ELSA [10] uses two-server distributed trust and secure computation to provide protection against malicious clients and a malicious server, while retaining practical end-to-end performance. PINE [11] focuses specifically on efficient exact verification of Euclidean norm bounds for secret-shared vectors, reducing the communication overhead associated with previous approaches. Flamingo [8] addresses the multi-round nature of FL and eliminates per-round setup costs present in earlier single-round designs. More recently, Aion [6] has investigated single-mask multi-round aggregation with malicious-participant tolerance, while Lighthouse [4] targets scalable single-server aggregation with constant server-committee communication independent of both the number of clients and input dimension.

Main Research Gap. Despite substantial progress in secure aggregation and privacy-preserving input validation, the interaction between the algebraic representation of lattice-based aggregation and the concrete cost of zero-knowledge validation remains relatively underexplored from an implementation perspective. Existing RLWE-based secure aggregation demonstrates that lattice structure can provide efficient aggregation and post-quantum security, while systems such as RoFL [2], ACORN [1], and PINE [11] demonstrate the importance of validating high-dimensional client inputs. However, these directions leave an implementation question that is particularly relevant when constructing an end-to-end post-quantum FL system: how should the RLWE computation itself be represented inside a proof system, and what are the resulting trade-offs among proof size, proving time, verification time, and aggregation cost?

Our work addresses this question by implementing the RLWE aggregation and validation pipeline using a C++/LibTorch machine-learning layer and a Rust cryptographic layer, and by experimentally comparing NTT/vector and RLWE-matrix R1CS representations.

3 System and Threat Model

3.1 Federated-learning setting

Consider a federation consisting of n clients and an aggregation server. Client i owns a private dataset

$$D_i = \{(x_{ij}, z_{ij}, y_{ij})\}_{j=1}^{d_i},$$

where z_{ij} denotes an optional protected or fairness-relevant attribute. The client performs local training and obtains a model update

$$m_i \in \mathbb{R}^N.$$

Because cryptographic operations operate over finite rings, the update is converted into a fixed-point integer representation:

$$\tilde{m}_i = \lfloor \Delta m_i \rfloor \bmod q,$$

where Δ is a publicly specified scaling factor and q is the RLWE modulus. Hence, the objective of secure aggregation is to recover

$$M = \sum_{i \in S} \tilde{m}_i$$

for the accepted participant set S , without revealing any individual $\tilde{m}_i$.

3.2 Protocol Flow (Figure 1)

1. Setup and Key Exchange
 - *Public Parameters:*
 - **Setup for Pedersen:** All n clients agree on a large prime q , and two generator elements g, h of an elliptic curve group $\mathbb{G}$.
 - **Setup for RLWE:** All n clients agree on the RLWE parameters $\text{pp} = \{N, q, T, \chi_s, \chi_e\}$ where N is the lattice dimension, q is the prime modulus, T is the plaintext modulus.
 - **Note:** These public RLWE parameters are shared among all parties using XOF hash function (SHAKE128)
 - *Key Agreement:* Each pair of clients (i, j) , where $0 \leq i, j \leq (n - 1)$, establish a shared secret $p_{i,j}$ via Diffie-Hellman key exchange.
 - *Mask Generation:* Client i generated a random zero-sum mask $\delta_i = \sum_{j>i} p_{i,j} - \sum_{j<i} p_{i,j}$, such that $\sum_{i=0}^{n-1} \delta_i = 0$.
2. Local Computation and Commitment
 - *PRG-Masking:* Each client (i) adds the random mask to their local secrets s_i to compute

$$s'_i = s_i + \delta_i \bmod q$$

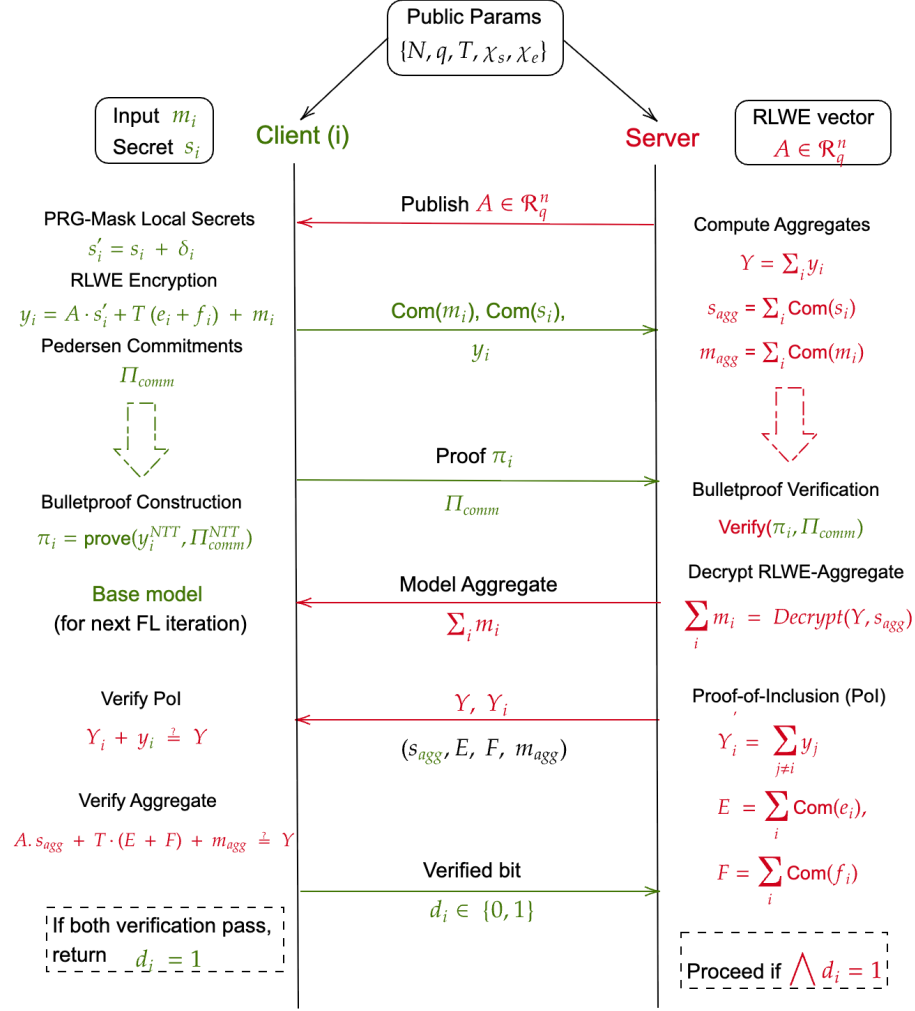


Fig. 1. Federated Training Round Overview

- *Pedersen Commitment*: Each client (i) commits to their locally computed update x_i :

$$\text{commit}(x_i) = g^{x_i} \cdot h^{t_i} \quad (1)$$

where t_i is a random blinding factor. Similarly, each client also computes $\text{commit}(s_i)$ and commitments to other private inputs.

- *RLWE Encryption*: The message x_i is encrypted as an RLWE plaintext $m_i \in R_T$, and encrypted as described in Section 5

$$y_i = a \cdot s_i + m_i + T \cdot (e_i + f_i) \mod q \quad (2)$$

where $s_i \leftarrow \chi_s$ is the secret key and $e_i, f_i \leftarrow \chi_e$ are the gaussian error vectors, constituting the RLWE private key $(s_i, e_i, f_i) \in (R_q)^3$.

- *Transmission*: Client sends Pedersen commitments (Π_{comm}) of all private inputs to the Server.

3. Global aggregation

- *Secret Key Aggregation*: $s_{agg} = \sum_{i=0}^{n-1} s'_i = \sum_{i=0}^{n-1} s_i$ since $\sum_{i=0}^{n-1} \delta_i = 0$.
- *Client Input Aggregation*: Homomorphic addition of the client-commitments from Equation 2:

$$y_{agg} = \sum_{i=0}^{N-1} y_i = a \sum_{i=0}^{N-1} s_i + \sum_{i=0}^{N-1} m_i + T \cdot \left(\sum_{i=0}^{N-1} e_i + \sum_{i=0}^{N-1} f_i \right)$$

- *RLWE Decryption* to get model aggregate of client updates $\sum_i m_i$.

4. Bulletproof for Valid Encryption: (details TBA)

5. Proof-of-Inclusion

- *Server Action*: The server publishes Y and the proof-of-inclusion $Y_i = \sum_{j \neq i} y_j$
- *Client Verification*: The proof of membership is valid if $Y_i + y_i = Y$, verified by each client i .
- *Server verification*: The server uses individual client openings to compute $E = \sum_{i=0}^{N-1} e_i$ and $F = \sum_{i=0}^{N-1} f_i$. It sends the aggregate opening (s_{agg}, E, F) to the client for final verification

$$a \cdot s_{agg} + T \cdot (E + F) + m_{agg} = Y_{agg}$$

3.3 Protocol Outline

Let client i hold a private local dataset

$$D_i = \{(x_{ij}, z_{ij}, y_{ij})\}_{j=1}^{n_i},$$

where x_{ij} denotes the input features, z_{ij} a potentially sensitive group attribute, and y_{ij} the corresponding task label. After local training, the client obtains a model update $m_i \in \mathbb{R}^N$, which is encoded into the RLWE plaintext domain. We denote the encoded vector by $\tilde{m}_i \in R_q$, where R_q is the underlying polynomial ring. Public system parameters include the modulus q , polynomial dimension N , noise distribution χ , and scaling/encoding parameters.

The protocol proceeds in five logical stages.

1. *Local update and zero-knowledge validation.* Client i first derives its local update m_i and encodes it into the representation used by the RLWE scheme. The client samples a secret (or masking) value s'_i and error terms e_i, f_i and constructs the ciphertext

$$y_i = As'_i + T(e_i + f_i) + \tilde{m}_i \in R_q, \quad (3)$$

where A and T are public system parameters. The exact distribution and representation of these quantities are specified by the underlying RLWE instantiation.

The client additionally generates Pedersen commitments to values that must later be bound to the proof and aggregate:

$$C_{m_i} = \text{Com}(\tilde{m}_i; r_i^m), \quad C_{s_i} = \text{Com}(s_i; r_i^s). \quad (4)$$

A zero-knowledge proof π_i^{val} is generated for the prescribed validity relation. Abstractly, the relation can be written as

$$\mathcal{R}_{\text{val}} = \{(y_i, \tilde{m}_i, s'_i, e_i, f_i) : y_i = As'_i + T(e_i + f_i) + \tilde{m}_i \wedge \tilde{m}_i \in \mathcal{M}\}, \quad (5)$$

where $\mathcal{M}$ denotes the permitted encoded update space and may include application-specific range or norm constraints. The client proves knowledge of a witness satisfying $\mathcal{R}_{\text{val}}$ without revealing the update itself.

2. *Private group information and fairness statistics.* When fairness auditing is enabled, client i does not directly reveal its sensitive group attribute g_i . Instead, it forms a commitment

$$C_{g_i} = \text{Com}(g_i; r_i^g) \quad (6)$$

and may provide a zero-knowledge proof π_i^{grp} that g_i belongs to the permitted group domain $\mathcal{G}$. This permits group membership to remain hidden while still binding the client to a well-defined group.

The client can also compute local evaluation statistics such as

$$(TP_i, FP_i, TN_i, FN_i),$$

or other application-specific contribution statistics. These values are encoded into a committed statistic object

$$C_{\text{stats},i} = \text{Com}(TP_i, FP_i, TN_i, FN_i, w_i; r_i^{\text{stats}}), \quad (7)$$

where w_i denotes an optional contribution or aggregation weight. Only the information required by the selected fairness audit is revealed or proven.

3. *Server-side verification and aggregation.* Client i sends

$$\text{Msg}_i = (y_i, C_{m_i}, C_{s_i}, C_{g_i}, \pi_i^{\text{val}}, \pi_i^{\text{grp}}, C_{\text{stats},i}, w_i) \quad (8)$$

to the server. The server first verifies the available zero-knowledge proofs. Only updates satisfying the prescribed validation relation are accepted.

Let $\mathcal{S} \subseteq \mathcal{C}$ denote the set of accepted clients. The aggregate ciphertext is

$$\gamma = \sum_{i \in \mathcal{S}} y_i. \quad (9)$$

By linearity of the RLWE construction,

$$\gamma = A \sum_{i \in \mathcal{S}} s'_i + T \sum_{i \in \mathcal{S}} (e_i + f_i) + \sum_{i \in \mathcal{S}} \tilde{m}_i. \quad (10)$$

Consequently, the aggregate plaintext corresponds to the sum of the validated client updates, subject to the correctness and noise conditions of the underlying RLWE construction.

For each group $g \in \mathcal{G}$, the server can similarly form

$$\gamma_g = \sum_{i \in \mathcal{S}_g} y_i, \quad \mathcal{S}_g = \{i \in \mathcal{S} : g_i = g\}, \quad (11)$$

and

$$\gamma = \sum_{g \in \mathcal{G}} \gamma_g. \quad (12)$$

The corresponding committed fairness statistics are aggregated homomorphically:

$$C_{\text{stats}} = \sum_{i \in \mathcal{S}} C_{\text{stats}, i}. \quad (13)$$

4. Proof of inclusion. For an accepted client i , define the aggregate excluding that client as

$$\gamma_{-i} = \sum_{\substack{j \in \mathcal{S} \\ j \neq i}} y_j. \quad (14)$$

The fundamental inclusion relation is

$$\gamma_{-i} + y_i = \gamma. \quad (15)$$

A PoI protocol can provide the client with γ_{-i} together with the information required to verify Eq. (15). At the group level, the corresponding consistency relation is

$$\gamma = \sum_{g \in \mathcal{G}} \gamma_g, \quad (16)$$

which allows an auditor to verify that the reported group-level decomposition is consistent with the aggregate.

5. Decryption and fairness auditing. After successful verification, the aggregate ciphertext is decrypted using the aggregate secret/masking material according to the underlying RLWE construction:

$$M = \text{Dec}(\gamma, s'_{\text{agg}}), \quad s'_{\text{agg}} = \sum_{i \in \mathcal{S}} s'_i. \quad (17)$$

The resulting aggregate update is used to construct the next global model.

The same protocol transcript provides inputs for inclusion fairness auditing. Let E_g denote the number of eligible clients in group g , and let I_g denote the number of accepted/included clients. The group inclusion rate is

$$CR_g = \frac{I_g}{E_g}. \quad (18)$$

Useful audit statistics include the inclusion disparity

$$\Delta_{\text{incl}} = \max_{g \in \mathcal{G}} CR_g - \min_{g \in \mathcal{G}} CR_g, \quad (19)$$

the inclusion disparate-impact ratio

$$DIR_{\min} = \frac{\min_{g \in \mathcal{G}} CR_g}{\max_{g \in \mathcal{G}} CR_g}, \quad (20)$$

and Jain's inclusion fairness index

$$J_{\text{incl}} = \frac{\left(\sum_{g \in \mathcal{G}} CR_g \right)^2}{|\mathcal{G}| \sum_{g \in \mathcal{G}} CR_g^2}. \quad (21)$$

These quantities describe the fairness of client inclusion; they do not by themselves constitute a cryptographic proof that the resulting ML model is fair.

3.4 Security Objectives

The proposed protocol is designed to provide four complementary security and accountability objectives.

O1: Privacy of Input. The protocol protects each client's individual model update m_i from the aggregation server and other clients. The server should learn only the permitted aggregate information and no additional information about any individual update.

O2: Verifiable Client Integrity. Each accepted client must demonstrate, in zero knowledge, that its encrypted update satisfies the prescribed validity relation. In particular, the proof binds the committed update to the corresponding RLWE computation and may additionally enforce application-level constraints such as $\|m_i\|_2 \leq B_2$ and $\|m_i\|_\infty \leq B_\infty$.

O3: Aggregate Inclusion Integrity. A client that receives an inclusion certificate should be able to verify that its valid contribution is consistently represented in the server’s aggregate. For client i , the proof-of-inclusion relation requires

$$\gamma_{-i} + y_i \stackrel{?}{=} \gamma, \quad (22)$$

where γ is the claimed aggregate and γ_{-i} is the aggregate of the remaining accepted contributions.

O4: Fairness Auditability. The protocol provides cryptographically verifiable information from which participation and contribution disparities across pre-defined groups can be audited without revealing individual client updates or, where required, individual group membership. For group g , the inclusion rate is defined as

$$CR_g = \frac{I_g}{E_g}, \quad (23)$$

where I_g and E_g denote the numbers of included and eligible clients in group g , respectively. These statistics can be used to evaluate inclusion disparity and established downstream machine-learning fairness measures.

3.5 Threat Model

We consider a federated learning environment in which the aggregation server and clients may have different security goals. The primary model assumes a *semi-honest-but-curious server*: the server follows the protocol but attempts to infer information about individual client updates, sensitive group membership, or other private statistics from the messages it receives. The server is assumed not to break the cryptographic assumptions underlying the RLWE encryption, Pedersen commitments, or zero-knowledge proof system.

Clients, in contrast, may be *malicious*. A malicious client may submit an arbitrary model update, construct malformed ciphertexts, attempt to violate the prescribed update bounds, or provide inconsistent auxiliary statistics. The zero-knowledge validation mechanism is intended to prevent an invalid update from being accepted unless the client can produce a valid proof for the specified relation. Pedersen commitments additionally bind committed values to the corresponding proofs and aggregate statistics.

The protocol also considers an *aggregate-inclusion adversary* who attempts to cause a valid contribution to be omitted from, or misrepresented within, the claimed aggregate. The PoI relation in Eq. (15) provides an explicit consistency check between an individual contribution and the aggregate. At the group level, Eq. (16) provides the corresponding consistency condition for a reported group decomposition.

We assume authenticated and integrity-protected communication between clients and the server. This prevents an external network adversary from silently modifying protocol messages. We do not, however, assume that communication integrity alone prevents a malicious server from reporting an incorrect aggregate;

such claims require the aggregate verification and PoI mechanisms described above.

The protocol does not assume that cryptographic verification alone implies fairness of the learned model. In particular, inclusion fairness and model-level fairness are distinct properties. Model-level metrics such as statistical parity, equal opportunity, equalized odds, worst-group accuracy, or worst-group loss depend additionally on the data distribution, learning algorithm, model, and evaluation procedure. The cryptographic protocol instead provides integrity and auditability for the information from which such properties can be evaluated.

3.6 Computational and Communication Cost Metrics

We compare communication and computational cost of computing and transmitting the Pedersen Commitments (Equation (1)) vs RLWE-based commitments (Equation (2)):

4 Preliminaries

Notations. Let N be a power-of-two and q be a positive integer such that $q \equiv 1 \pmod{N}$. Then the N -th cyclotomic polynomial is given by $\Phi_N(x) = x^N + 1$. Let $R = \mathbb{Z}[X]/(X^N + 1)$ be the cyclotomic ring of polynomials. An element $a = \sum_{i=0}^{N-1} a_i x^i$ in R is a polynomial of degree at most $(N-1)$ where $a_i \in \mathbb{Z}$. The residual/quotient ring of R modulo q is by $R_q = \mathbb{Z}_q[X]/(X^N + 1)$, which is isomorphic to R/qR . Similarly, an element $a = \sum_{i=0}^{N-1} a_i x^i$ in R_q is a polynomial of degree at most $(N-1)$ where $a_i \in \mathbb{Z}_q$. The coefficient embedding of a polynomial $a = \sum_{i=0}^{N-1} a_i x^i \in R_q$ is the vector $\mathbf{a} = (a_0, a_1, \dots, a_{N-1})$.

4.1 Number Theoretic Transform (NTT)

The number theoretic transform (NTT) provides an efficient way to multiply two polynomials in R_q . We choose N and q such that $q \equiv 1 \pmod{2N}$ for an efficient implementation of the number theoretic transform. Let ω be a primitive N -th root of unity in $\mathbb{Z}_q$, i.e., $\omega^N \equiv 1 \pmod{q}$. For a vector $\mathbf{a} = (a_0, a_1, \dots, a_{N-1})$, the NTT transformation is given by $A_i = \sum_{j=0}^{N-1} a_j \omega^{ij} \pmod{q}$ for $i = 0, 1, \dots, N-1$. We denote this by $\mathbf{A} = \text{NTT}(\mathbf{a})$. The inverse NTT is given by $a_i = \frac{1}{N} \sum_{j=0}^{N-1} A_j \omega^{-ij} \pmod{q}$, $i = 0, 1, \dots, N-1$, which is denoted by $\mathbf{a} = \text{INTT}(\mathbf{A})$. Given two polynomials $a, b \in R_q$, computing $c = a \cdot b \pmod{x^N + 1}$ can be done efficiently by exploiting the negative wrapped convolution [?]. Let ψ be a primitive $2N$ -th root of unity in $\mathbb{Z}_q$ such that $\psi^2 = \omega$ as $\psi^{2N} = (\psi^2)^N = \omega^N = 1$. Given two polynomials $a, b \in R_q$, $c = a \cdot b \pmod{x^N + 1}$ can be computed as

$$c = (1, \psi^{-1}, \psi^{-2}, \dots, \psi^{-(N-1)}) \circ \text{INTT}(\text{NTT}(\hat{\mathbf{a}})) \circ \text{INTT}(\hat{\mathbf{b}})$$

where $\hat{\mathbf{a}} = (a_0, \psi^{-1}a_1, \psi^{-2}a_2, \dots, \psi^{-(N-1)}a_{N-1})$ is computed from $\mathbf{a}$, $\hat{\mathbf{b}} = (b_0, \psi^{-1}b_1, \psi^{-2}b_2, \dots, \psi^{-(N-1)}b_{N-1})$ is computed from $\mathbf{b}$ and $\circ$ is a component-wise dot product.

4.2 Commitments

Crypto primitives to be formally defined:

- commitment schemes: Pedersen and Lattice-based
- secure aggregation (PRG-masking)

Lattice-based commitments Compare costs for $k = 1$ (plain RLWE) and $k > 1$ (Module RLWE of dimension k) with the lattice dimension $n \in \{4096, 8192\}$ and the corresponding prime modulus q (155-bit prime).

Pedersen commitments We choose the NIST P-256 Elliptic curve (128-bit security) for our implementation of Pedersen commitments.

4.3 Zero Knowledge Proofs: Bulletproof

4.4 Fairness in ML: Individual and Group

5 RLWE-based Secure Aggregation

Ring Learning With Errors (RLWE). The RLWE assumption ([7]) is parameterized by the quotient ring R_q of polynomials having degree upto N , defined over coefficients in $\mathbb{Z}_q$, where $q > 0$ is a large prime integer modulus. The assumption states that for any secret vector $s \leftarrow \chi_s$ picked from a distribution over R , given $\ell = \text{poly}(N)$ many independent samples from the following distribution

$$\mathcal{D}_{N,q,\chi_e}(s) = \{(a, as + e) \in (R_q)^2 \mid a \leftarrow R_q, e \leftarrow \chi_e\},$$

it is computationally hard to distinguish these samples from ℓ samples chosen uniformly at random over R_q . This hardness assumption forms the underlying security guarantee for the RLWE-based encryption scheme defined as follows.

RLWE encryption. We consider an RLWE-based encryption scheme from [9], which is also considered in [1] (Usenix 2023). To encrypt a plaintext message $m \in R_T$, we sample a secret key $s \leftarrow \chi_s$ and an error $e \leftarrow \chi_e$ and compute

$$\text{Encode}(s, m) = (a, a \cdot s + m + T \cdot e)$$

where $a \leftarrow R_q$. It is easy to check that this encryption scheme has the following additive homomorphic property on the secret key and the plaintext:

$$\text{Encode}(s_1, m_1) + \text{Encode}(s_2, m_2) = \text{Encode}(s_1 + s_2, m_1 + m_2).$$

RLWE-based secure aggregation. We define an RLWE-based secure aggregation (SA) protocol, consisting of a tuple of four algorithms

$$\text{SA} = (\text{KeyGen}, \text{Encode}, \text{Decode}, \text{Aggregate}).$$

- $(\mathbf{pp}, \{s_i\}_{i=1}^n) \leftarrow \text{KeyGen}(1^\lambda, n)$: On a security parameter λ and the number of users participating in the aggregation process, sample public parameters $\mathbf{pp} = \{N, q, T, \chi_s, \chi_e\}$. For each user $\mathcal{U}_i$, sample a secret key s_i from χ_s for $i \in [1, n]$. It also computes $s_0 = -\sum_{i=1}^n s_i$ and outputs $(\mathbf{pp}, s_i), i \in [1, n]$ to each U_i and $(\mathbf{pp}, s_0)$ to the aggregator.
- $y_i \leftarrow \text{Encode}(s_i, m_i, \Delta t)$: To encrypt m_i using s_i , it generates $a \leftarrow \text{SHAKE128}(\Delta t)$, samples $e_i \leftarrow \chi_e$ and computes

$$y_i = a \cdot s_i + m_i + T \cdot (e_i + f_i) \in R_q.$$

An RLWE ciphertext is $c_i = (a, y_i)$.

- **Note:** We propose a distributed **KeyGen** mechanism where we do not require a trusted setup. Our idea is that each user samples $s_i \leftarrow \chi_s$, and then they run a masking PRG-based algorithm to compute $s_0 = -\sum_{i=1}^n s_i \bmod q$.

6 Zero Knowledge Input Validation

6.1 NTT Representation of RLWE-based SA

We represent an RLWE-based ciphertext in the NTT domain, meaning each user stores ciphertext y_i in the NTT domain, which is $\text{NTT}(\mathbf{c}_i) = (\text{NTT}(\mathbf{a}), \text{NTT}(\mathbf{y}_i))$

$$\mathbf{Y}_i = \text{NTT}(\mathbf{y}_i) = \text{NTT}(\mathbf{a}) \circ \text{NTT}(\mathbf{s}_i) + \text{NTT}(\mathbf{m}_i) + T \cdot \text{NTT}(\mathbf{e}_i)$$

For secure aggregation, the aggregated ciphertext y in the NTT domain, denoted by $\text{NTT}(\mathbf{Y})$, can be computed as

$$\text{NTT}(\mathbf{Y}) = \text{NTT}(\mathbf{a}) \circ \sum_{i=1}^n \text{NTT}(\mathbf{s}_i) + \sum_{i=1}^n \text{NTT}(\mathbf{m}_i) + T \cdot \sum_{i=1}^n \text{NTT}(\mathbf{e}_i).$$

The aggregated ciphertext in the polynomial domain can be computed as $c = (a, y)$ where $y = \sum_{i=1}^n y_i$ is computed as $a = \text{INTT}(\mathbf{A})$ and $y = \text{INTT}(\mathbf{Y})$.

6.2 R1CS Arithmetic Circuit Representation

6.3 Proof of Valid Encryption: Bulletproof for R1CS

7 Proof of Inclusion and Fairness

Cryptographic inclusion and statistical fairness are distinct properties. The proposed **proof-of-inclusion** mechanism establishes that a client's valid contribution is consistently represented in the server's aggregate; it does not, by itself, imply demographic parity, equality of opportunity, or equalized odds of the resulting machine-learning model. These properties depend on the distribution of training data, model architecture, learning algorithm, and evaluation population. We therefore use PoI as an integrity primitive that enables trustworthy measurement of participation and contribution disparities, while evaluating downstream model fairness using established group-level metrics.

Formal fairness metrics (to be detailed):

1. Inclusion rate
2. Inclusion disparity
3. Inclusion disparate-impact ratio
4. Jain’s fairness index

8 Security Analysis

To add formal proofs of key security objectives from section 3.4

9 Implementation

The implementation is deliberately divided between a systems layer and a cryptographic layer. Local model training, tensor manipulation, fixed-point encoding, and protocol orchestration are implemented in C++ using LibTorch, while the RLWE arithmetic, NTT operations, Pedersen commitments, and Bulletproof R1CS proofs are implemented in Rust using curve25519-dalek and bulletproofs. This separation permits the machine-learning implementation to operate on conventional tensor representations while isolating security-critical cryptographic operations inside a memory-safe implementation. The resulting architecture also enables direct measurement of the cryptographic overhead introduced by input validation rather than treating the proof system as an abstract component.

A central implementation result is that the representation of the validity relation has a substantial effect on the resulting Bulletproof. We evaluate two R1CS formulations: an NTT/vector formulation and an RLWE-matrix formulation. The former expresses the relation through element-wise multiplication constraints, whereas the latter exploits publicly known RLWE matrix coefficients through linear combinations. Consequently, the two formulations exhibit different proof-generation and verification costs despite representing closely related algebraic relations. In our implementation, the RLWE-matrix formulation produces a substantially smaller proof—approximately 417 bytes in the measured configurations—while the NTT/vector formulation produces proofs of 1121, 1185, and 1249 bytes for $N = 2048, 4096$, and 8192 , respectively. The measurements also expose an important trade-off: the compact RLWE-matrix proof incurs considerably greater proving and verification time because its current implementation performs explicit matrix-style operations.

Datasets MNIST [3] (28x28 grayscale) and CIFAR-10 [5] (32x32 RGB)

Training Models Suppose length of messages (according to CNN model parameters size), emphasize number of commitments required of each of the three kinds (for same sized messages).

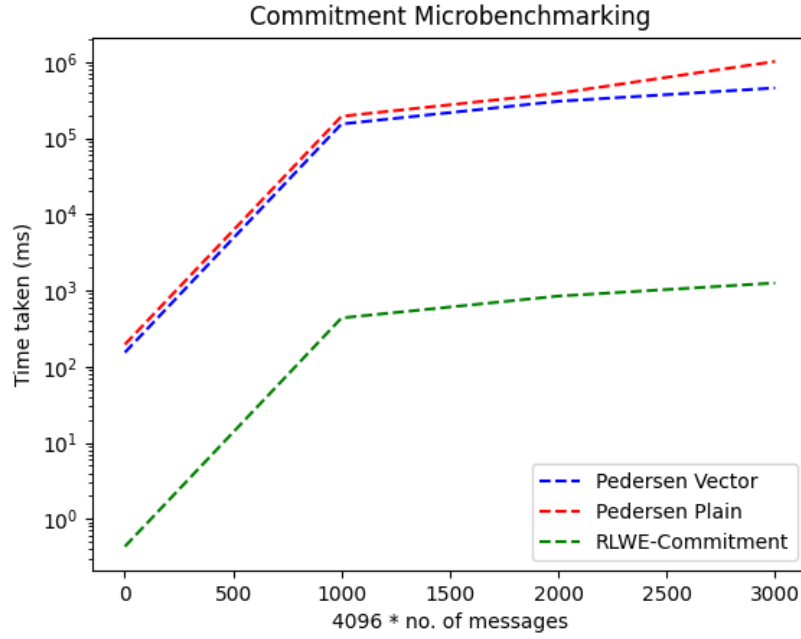


Fig. 2. Commitments Microbenchmarking

10 Experimental Evaluation

10.1 Cryptographic Commitments

We compare three kinds of commitment for sending local client updates for aggregation. Figure 2 shows a comparative analysis of RLWE-based commitments as compared to the different variants of Pedersen Commitments. We observe that although Pedersen commitments are faster for individual messages, but the RLWE-based commitment is most optimal for large batches of model data.

1. Pedersen Commitments are implemented in C++ using OpenSSL. Table 1 shows the benchmarking results for computing Pedersen commitments.
 - **Plain Pedersen** commitments which commit to each individual message separately.
 - **Vector Pedersen** commitments (vector dimension $n=4096$) capture n messages in each commitment, averaging to 153 ms (Table 2), which is 0.78x (21 percent faster) as compared to 195 ms for 4096 individual commitments (Table 1).
2. RLWE commitments are implemented in C++ using the Microsoft SEAL API. Table 3 shows the benchmarking results for RLWE-based commitments

message length	total time	average per commitment
4096 commits	195.498 ms	0.047 ms
4096000 commits	193363 ms	0.047 ms
8192000 commits	388270 ms	0.047 ms
12288000 commits	1016500 ms	0.082 ms

Table 1. Plain Pedersen (single-message commit)

number of operations	total time	average per commitment
1000 commits	153343 ms	153.343 ms
2000 commits	304584 ms	152.292 ms
3000 commits	455284 ms	151.761 ms

Table 2. Vector Pedersen ($n = 4096$)

(Equation (2)) for $n = 4096$ i.e each commitment corresponding to 4096 elements of the message to be communicated (local client updates). Here, we see a significant improvement in computational cost as each commitment takes 0.435 ms, which is 444x faster than the 195 ms for plain pedersen commitments, and 352x faster than the 153 ms for vector pedersen commitments of messages having the same length.

number of operations	total time	average per commitment
1000 commitments	435.421 ms	0.435 ms
2000 commitments	843.919 ms	0.422 ms
3000 commitments	1253.33 ms	0.418 ms

Table 3. RLWE-Commitments Benchmarking

10.2 Bulletproof Implementation

We record proof construction and verification times along with proof sizes (averaged over 100 iterations with random fixed-length inputs (generated from uniform probability distributions) and error vectors in Table 5 and compare it to equivalent results for the ACORN [1] cite Bulletproof implementation in Table 4.

Negacyclic matrix-based RLWE Encoding ([1])

NTT-based RLWE Encoding (This result)

Input length (N)	Proof time	Proof size	Verification time
2048	17.87 s	417 B	14.34 s
4096	65.55 s	417 B	58.37 s
8192	271.13 s	417 B	257.22 s

Table 4. Proof-of-Valid-Encoding (RLWE Matrix)

Input length (N)	Proof time	Proof size	Verification time
2048	8.29 s	1121 B	770.7 ms
4096	16.45 s	1185 B	1.53 s
8192	33.86 s	1249 B	3.06 s

Table 5. Proof-of-Valid-Encoding (NTT-R1CS)

Observations: Despite its higher computational cost, the RLWE-matrix gadget produces a remarkably compact proof. Its proof size remains constant at 417 bytes for all three input lengths, whereas the NTT-vector gadget grows from 1121 bytes at $N=2048$ to 1249 bytes at $N=8192$. This is a consequence of the structure of the Bulletproof R1CS representation used in the [1] implementation, where the matrix coefficients are public constants and are incorporated directly into linear combinations; consequently, the gadget does not introduce R1CS multiplication gates for the matrix-vector product. The resulting proof therefore does not require the additional second-phase commitment components associated with multiplication gates, and its serialized size is independent of the number of matrix coefficients.

By contrast, the NTT-vector gadget introduces multiplication constraints between committed witness variables, causing the proof to contain additional commitment elements and a logarithmic inner-product component whose size increases with the circuit dimension. Thus, the results illustrate a fundamental trade-off: the RLWE-matrix formulation substantially reduces proof size at the cost of considerably higher prover and verifier computation due to its explicit matrix operations.

11 Conclusion

We presented a privacy-preserving and verifiable federated-learning architecture based on RLWE secure aggregation, zero-knowledge input validation, and proof-of-inclusion. The construction combines privacy of individual model updates with cryptographic validation of client inputs and enables clients to verify that their valid contributions are represented consistently in the aggregate. We further extended this mechanism toward fairness-aware aggregation by defining group-level inclusion statistics and privacy-preserving mechanisms for auditing participation disparities. This creates a distinction between two properties that are often con-

flated in federated learning: cryptographic integrity of participation and statistical fairness of the resulting model. The former can be established through the proposed PoI mechanism, while the latter is evaluated using established group- and client-level fairness metrics.

Our implementation demonstrates that the representation of the RLWE relation inside the zero-knowledge circuit is itself an important performance parameter. The RLWE-matrix representation produces substantially smaller Bulletproofs than the NTT/vector representation, at the cost of increased proving and verification time in the current implementation. The resulting system therefore exposes an explicit trade-off among proof size, computation, aggregation efficiency, and fairness-auditability. Future work will investigate more efficient group-statistic proofs, multi-round fairness guarantees, and circuit-level optimization of the RLWE-matrix formulation.

References

1. Bell, J., Gascón, A., Lepoint, T., Li, B., Meiklejohn, S., Raykova, M., Yun, C.: ACORN: Input validation for secure aggregation. In: 32nd USENIX Security Symposium (USENIX Security 23). pp. 4805–4822. USENIX Association, Anaheim, CA (2023), <https://www.usenix.org/conference/usenixsecurity23/presentation/bell>
2. Burkhalter, L., Lycklama, H., Viand, A., Küchler, N., Hithnawi, A.: Rofl: Attestable robustness for secure federated learning. CoRR **abs/2107.03311** (2021), <https://arxiv.org/abs/2107.03311>
3. Cohen, G., Afshar, S., Tapson, J., van Schaik, A.: EMNIST: an extension of MNIST to handwritten letters. CoRR **abs/1702.05373** (2017), <http://arxiv.org/abs/1702.05373>
4. Garg, S., Kavousi, A., Kolonelos, D., Tairi, E., Wang, Z.: Lighthouse: Single-Server secure aggregation with $\mathcal{O}(1)$ Server-Committee communication at scale. In: 35th USENIX Security Symposium (USENIX Security 26). pp. 2983–3002. USENIX Association, Baltimore, MD (Aug 2026), <https://www.usenix.org/conference/usenixsecurity26/presentation/garg-sanjam>
5. Krizhevsky, A., Hinton, G.: Learning multiple layers of features from tiny images. Tech. Rep. 0, University of Toronto, Toronto, Ontario (2009), <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>
6. Liu, Y., Jia, Z., Chen, X., Bian, S., Xu, R., Li, D., Lu, Y.: Aion: Robust and efficient Multi-Round Single-Mask secure aggregation against malicious participants. In: 34th USENIX Security Symposium (USENIX Security 25). pp. 3025–3044. USENIX Association, Seattle, WA (Aug 2025), <https://www.usenix.org/conference/usenixsecurity25/presentation/liu-yizhong>
7. Lyubashevsky, V., Peikert, C., Regev, O.: On ideal lattices and learning with errors over rings. In: Advances in Cryptology – EUROCRYPT 2010. Lecture Notes in Computer Science, vol. 6110, pp. 1–23. Springer (2010). https://doi.org/10.1007/978-3-642-13190-5_1
8. Ma, Y., Woods, J., Angel, S., Polychroniadou, A., Rabin, T.: Flamingo: Multi-round single-server secure aggregation with applications to private federated learning. In: 2023 IEEE Symposium on Security and Privacy (SP). p. 477–496. IEEE (May 2023). <https://doi.org/10.1109/sp46215.2023.10179434>, <http://dx.doi.org/10.1109/SP46215.2023.10179434>

9. Martínez, R., Morillo, P., Rovira, S.: Implementation and performance of a rlwe-based commitment scheme and zkpok for its linear and multiplicative relations. *Cryptology ePrint Archive* (2023)
10. Rathee, M., Shen, C., Wagh, S., Popa, R.A.: Elsa: Secure aggregation for federated learning with malicious actors. In: 2023 IEEE Symposium on Security and Privacy (SP). pp. 1961–1979 (2023). <https://doi.org/10.1109/SP46215.2023.10179468>
11. Rothblum, G.N., Omri, E., Chen, J., Talwar, K.: PINE: Efficient verification of a euclidean norm bound of a Secret-Shared vector. In: 33rd USENIX Security Symposium (USENIX Security 24). pp. 6975–6992. USENIX Association, Philadelphia, PA (Aug 2024), <https://www.usenix.org/conference/usenixsecurity24/presentation/rothblum>